The Islamia University of Bahawalpur
Department of Information Technology

SOFTWARE SYSTEM DESIGN
(SSD)
GLOBAL EXPLORER
A Comprehensive Country Information Portal

Submitted by	Alina Farooq
Roll No	S25BINFT7M04067
Section	7th 1M
Supervisor	Prof. Dr. Dost Muhammad Khan
Subject	Final Year Project
Document Version	1.0
Date	June 10, 2026

Academic Year 2024–2025

1. Introduction

1.1 Purpose

This Software System Design (SSD) document provides a comprehensive description of the architectural, structural, and detailed design of Global Explorer — a web-based country information portal. It translates the requirements established in the SRS into concrete design decisions covering system architecture, component interfaces, data models, module interactions, and deployment strategy. This document serves as the technical blueprint for developers implementing the system.

1.2 Scope

Global Explorer is a client-side single-page application (SPA) built with React.js. It fetches country data from the publicly available REST Countries API and presents it through an intuitive, responsive interface. This SSD covers all major design aspects including:

- System architecture and component decomposition
- Detailed module and interface design
- Data flow and state management strategy
- Database / storage design (browser local storage schema)
- UI/UX design principles and wireframe descriptions
- Security, performance, and error handling design
- Deployment and build pipeline design

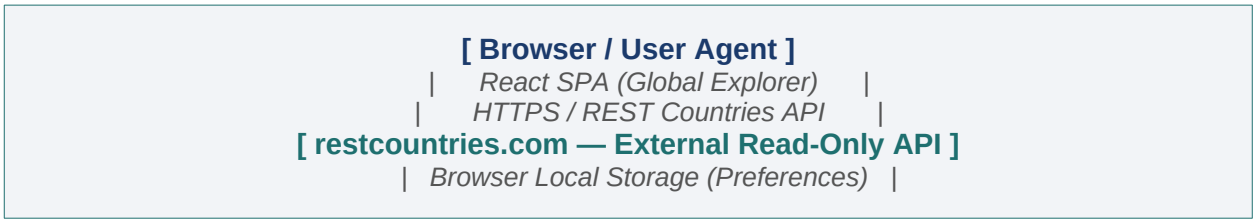
1.3 Definitions and Abbreviations

Term / Acronym	Definition
SSD	Software System Design
SRS	Software Requirements Specification
SPA	Single Page Application
CSR	Client-Side Rendering
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
UI	User Interface
UX	User Experience
CSS	Cascading Style Sheets
DOM	Document Object Model
HOC	Higher-Order Component

2. System Overview

2.1 System Context

Global Explorer operates entirely in the user's browser. There is no proprietary backend server. The system interacts with one external service — the REST Countries API — over HTTPS to retrieve country data. The browser's local storage acts as a lightweight persistence layer for user preferences.



NOTE: No server-side code is written or deployed. The system context diagram above illustrates the three-node interaction.

2.2 Design Goals

#	Goal	Design Rationale
G-01	**Modularity**	Decompose into small, reusable React components for maintainability.
G-02	**Performance**	Minimise API round-trips; cache responses in component state.
G-03	**Responsiveness**	Mobile-first CSS with breakpoints at 480px, 768px, and 1024px.
G-04	**Usability**	Consistent layout patterns; WCAG 2.1 AA colour contrast.
G-05	**Robustness**	Graceful error boundaries; fallback UI on API failure.
G-06	**Scalability**	Feature-flag pattern allows new data sources without refactoring.

3. Architectural Design

3.1 Architectural Pattern

Global Explorer adopts a Layered Component Architecture within the CSR/SPA paradigm. The architecture is divided into four horizontal layers:

Layer	Name	Responsibility
L-1	**Presentation Layer**	React components, JSX templates, CSS styling, routing.
L-2	**Application / Logic Layer**	Custom hooks, context providers, business rules, filtering & sorting.
L-3	**API / Service Layer**	Axios/Fetch wrappers, endpoint constants, response transformers.
L-4	**Persistence Layer**	Local storage utilities for favorites and theme; read/write helpers.

3.2 Component Architecture Diagram

The top-level component hierarchy is structured as follows:

```
App  (Root)
 ├── Navbar
 │    ├── Logo
 │    ├── SearchBar
 │    ├── ThemeToggle
 │    └── NavLinks  (Home | Compare | Favorites)
 ├── Router (React Router v6)
 │    ├── /           → HomePage
 │    │    ├── HeroBanner
 │    │    ├── RegionFilter
 │    │    └── CountryGrid  → CountryCard[]
 │    ├── /country/:code  → DetailPage
 │    │    ├── FlagDisplay
 │    │    ├── InfoGrid
 │    │    └── BorderLinks
 │    ├── /compare  → ComparePage
 │    │    └── CompareTable
 │    └── /favorites  → FavoritesPage
 └── Footer
```

4. Module Design

4.1 Module Catalogue

Module ID	Module Name	Description
MOD-01	App	Root component; provides ThemeContext and Router. Manages global theme state.
MOD-02	Navbar	Top navigation: logo, search bar, theme toggle, route links.
MOD-03	HomePage	Landing page — fetches all countries, renders CountryGrid, handles region filter.
MOD-04	CountryCard	Reusable card displaying flag, name, capital, population, region. Links to DetailPage.
MOD-05	CountryGrid	Renders a paginated list of CountryCard items; accepts filtered data prop.
MOD-06	DetailPage	Full-profile page for a selected country. Fetches by alpha code. Displays all fields.
MOD-07	ComparePage	Allows selection of two countries; renders CompareTable side by side.
MOD-08	FavoritesPage	Reads favorites from local storage and renders saved CountryCard items.
MOD-09	SearchBar	Controlled input with debounced autocomplete suggestions list.
MOD-10	RegionFilter	Dropdown to filter country list by continent/region.
MOD-11	ThemeToggle	Switch toggling dark/light mode; writes preference to local storage.
MOD-12	apiService	Axios wrappers for all REST Countries API endpoints; centralised error handling.
MOD-13	storageService	Helper functions for reading/writing favorites and theme to local storage.
MOD-14	useCountries	Custom hook: fetches all countries, exposes filtered/sorted list.
MOD-15	ThemeContext	React Context providing theme value and toggle function app-wide.

4.2 Module Interface Specifications

4.2.1 apiService (MOD-12)

File: src/services/apiService.js

Function	Returns	Description
getAllCountries()	Promise<Country[]>	GET /all — returns array of all country objects.
getCountryByName(name)	Promise<Country[]>	GET /name/{name}?fullText=false.
getCountryByCode(code)	Promise<Country>	GET /alpha/{code} — used for detail page routing.

getCountriesByRegion(region)	Promise<Country[]>	GET /region/{region}.
getCountriesByCodes(codes[])	Promise<Country[]>	GET /alpha?codes=… — for border links.

4.2.2 storageService (MOD-13)

File: src/services/storageService.js

Function	Returns	Description
getFavorites()	string[]	Returns array of saved country alpha-3 codes from local storage.
addFavorite(code)	void	Appends code to favorites array in local storage.
removeFavorite(code)	void	Removes code from favorites array in local storage.
isFavorite(code)	boolean	Checks if a given code exists in favorites.
getTheme()	'light'|'dark'	Reads saved theme preference; defaults to 'light'.
setTheme(theme)	void	Writes theme preference to local storage.

5. Data Design

5.1 Country Data Object (from REST Countries API)

The primary data entity is the Country object as returned by the REST Countries API. Key fields used by Global Explorer are:

Field	Type	Description
name.common	string	Common English name (e.g., 'Germany').
name.official	string	Official formal name.
name.nativeName	object	Native names keyed by language code.
cca2	string	ISO 3166-1 alpha-2 two-letter code (e.g., 'DE').
cca3	string	ISO 3166-1 alpha-3 three-letter code (e.g., 'DEU').
flags.svg / png	string (URL)	Flag image URL.
capital	string[]	Array of capital city names.
region	string	Broad region (Africa, Americas, Asia, Europe, Oceania).
subregion	string	Finer regional grouping.
population	number	Total population count.
area	number	Total area in km².
languages	object	Languages keyed by code; values are language names.
currencies	object	Currencies keyed by code; each has name and symbol.
timezones	string[]	Array of IANA timezone strings.
borders	string[]	Alpha-3 codes of bordering countries.
latlng	number[]	[latitude, longitude] coordinates.
coatOfArms.svg	string (URL)	Coat of arms image URL (may be null).
maps.googleMaps	string (URL)	Google Maps link.
gini	object	Gini index keyed by year.

5.2 Local Storage Schema

Key	Type	Example Value	Notes
ge_favorites	JSON Array	["DEU","JPN","BRA"]	Array of alpha-3 codes.
ge_theme	string	"dark"	'light' or 'dark'.

NOTE: All keys are prefixed with 'ge_' to avoid conflicts with other apps sharing the same origin.

6. User Interface Design

6.1 Design Principles

- Mobile-first CSS; breakpoints at 480 px, 768 px, 1024 px, and 1280 px.
- CSS custom properties (variables) used for all colours to enable one-line theme switching.
- Minimum touch target size 44 × 44 px on all interactive elements.
- WCAG 2.1 AA contrast ratio (≥ 4.5:1 for normal text, ≥ 3:1 for large text).
- Skeleton loading screens replace spinners for content-heavy pages.
- Consistent spacing scale: 4 px base unit — 4, 8, 12, 16, 24, 32, 48, 64 px.

6.2 Page Wireframe Descriptions

6.2.1 Home Page

NavBar (top)	Logo | Search input | Theme Toggle | Nav Links
Hero Banner	Full-width colour strip with headline and large search bar
Filter Bar	Dropdown: All Regions | Africa | Americas | Asia | Europe | Oceania
Country Grid	Responsive CSS Grid (4 → 3 → 2 → 1 columns) of CountryCard components
Each Card	Flag image, Country name (bold), Capital, Population, Region badge
Footer	Project name, academic details, API credit

6.2.2 Country Detail Page

Top Row	Large flag image (left) + Official name + Region badge (right)
Stats Row	Population | Area | Density — three stat blocks
Info Grid	8-column grid: Capital, Region, Languages, Currencies, Timezones, Borders, TLD, Calling Code
Borders Section	Clickable chip buttons for each bordering country (alpha-3 → full name)
Favourites Button	Heart/bookmark icon — togglable; saved to local storage
Map Section	Embedded OpenStreetMap iframe centred on country coordinates

6.2.3 Compare Page

Selector Row	Two dropdowns — each shows autocomplete list of all countries
Compare Table	Two-column table, attribute labels on far left; one column per country
Compared Attrs	Name, Flag, Capital, Population, Area, Region, Languages, Currencies, Timezones

Highlight	Higher population / larger area highlighted in teal for quick visual differentiation

6.2.4 Favorites Page

Empty State	Illustration + 'No favorites yet. Start exploring!' CTA
Grid View	Same CountryCard grid as Home Page but filtered to saved countries only
Remove Button	Visible 'x' or trash icon on each card to remove from favorites

6.3 Theme Design Tokens

Token	Light Mode	Dark Mode	Usage
--bg-primary	#F7F6F2	#1A1D26	Page background
--bg-card	#FFFFFF	#252836	Card / panel background
--text-primary	#0F1117	#F0EEE8	Body text
--text-secondary	#6B6860	#9A9690	Muted labels
--accent	#1E6F6F	#2EAFAF	Brand teal
--border	#D8D4C8	#35394A	Dividers

7. Data Flow Design

7.1 Search Data Flow

The following sequence describes how a user search is processed:

Step	Actor / Component	Action
1	**User (SearchBar)**	Types 2+ characters into the search input.
2	**SearchBar**	Debounces input (300 ms) and triggers onChange callback.
3	**HomePage / useCountries hook**	Filters pre-fetched country list client-side for autocomplete.
4	**SearchBar**	Renders suggestion dropdown from filtered list.
5	**User**	Selects a suggestion or presses Enter.
6	**React Router**	Navigates to /country/{cca3} route.
7	**DetailPage**	Calls apiService.getCountryByCode(cca3).
8	**apiService**	Performs HTTPS GET to restcountries.com/v3.1/alpha/{code}.
9	**REST Countries API**	Returns JSON payload.
10	**DetailPage**	Updates local state; triggers re-render with country data.

7.2 Favorites Data Flow

Step	Actor / Component	Action
1	**User**	Clicks the heart/bookmark icon on a CountryCard or DetailPage.
2	**Component**	Calls storageService.addFavorite(cca3) or removeFavorite(cca3).
3	**storageService**	Reads ge_favorites from localStorage; updates array; writes back.
4	**Component**	Updates local isFavorite state; icon animates to reflect new state.
5	**FavoritesPage (on mount)**	Calls storageService.getFavorites() to load all saved codes.
6	**FavoritesPage**	Calls apiService.getCountriesByCodes(codes) for display data.

8. State Management Design

8.1 State Strategy

Global Explorer avoids a third-party state library (e.g., Redux). Instead it uses React's built-in hooks and Context API, sufficient for this application's complexity level:

State Type	Mechanism	What It Holds
Local Component State	useState	Loading flags, error messages, form inputs, selected filters.
Remote/Async State	useEffect + useState	API fetch results; status: idle | loading | success | error.
Global Theme State	ThemeContext (useContext)	Current theme ('light'|'dark'), toggle function.
Persistent State	localStorage via storageService	Favorites list, theme preference.
Derived State	useMemo	Filtered/sorted country lists derived from raw API data.

8.2 Custom Hooks

Hook	Signature & Purpose
useCountries	useCountries(region?) → { countries, loading, error } — fetches and caches all or regional countries.
useCountry	useCountry(code) → { country, loading, error } — fetches single country by alpha code.
useFavorites	useFavorites() → { favorites, add, remove, isFavorite } — syncs with localStorage.
useTheme	useTheme() → { theme, toggle } — wrapper around ThemeContext.
useDebounce	useDebounce(value, delay) → debouncedValue — delays rapid state updates (search input).

9. Error Handling Design

9.1 Error Categories and Responses

Error ID	Error Type	User-Facing Message	Recovery Mechanism
ERR-01	API Network Failure	Unable to connect. Please check your internet connection.	Retry button; display cached data if available.
ERR-02	API 404 Not Found	Country not found. Try a different name.	Clear search; show suggestions.
ERR-03	API 500 Server Error	The data service is temporarily unavailable.	Auto-retry after 5 seconds.
ERR-04	Malformed JSON Response	Unexpected data format received.	Log to console; show generic error card.
ERR-05	Local Storage Unavailable	Favorites cannot be saved in this session.	Degrade gracefully; disable favorites icon.
ERR-06	Empty Search Query	Please enter at least 2 characters.	Inline input validation message.

9.2 React Error Boundary

A top-level ErrorBoundary component wraps the Router. Any unhandled JavaScript error in a child component tree triggers the fallback UI, which displays a friendly error message and a 'Reload Application' button, preventing the entire app from going blank.

10. Security Design

Security ID	Concern	Design Decision
SEC-01	API Communication	All requests to REST Countries API are made over HTTPS. HTTP requests are blocked.
SEC-02	No Sensitive Data	No authentication tokens, passwords, or personal data are collected or stored.
SEC-03	Local Storage Scope	Only non-sensitive preferences (favorites, theme) are stored; no PII.
SEC-04	XSS Prevention	All dynamic country data is rendered via React's JSX (auto-escaping); no innerHTML.
SEC-05	CORS	The REST Countries API exposes CORS headers; no proxy server is needed.
SEC-06	Content Security Policy	CSP meta tag restricts scripts to 'self' and cdn.jsdelivr.net only.
SEC-07	Dependency Audit	npm audit run before every release; high-severity vulnerabilities block build.

11. Performance Design

Perf ID	Strategy	Implementation Detail
PERF-01	Single API Fetch on Load	getAllCountries() is called once on app mount; result cached in state to avoid repeated fetches.
PERF-02	Debounced Search	useDebounce(300ms) prevents an API call on every keystroke; filtering is client-side on cached data.
PERF-03	Lazy Route Loading	React.lazy() and Suspense defer loading of Compare and Favorites pages until first navigation.
PERF-04	Image Optimisation	Flag images loaded from API CDN with natural lazy loading attribute on <img> tags.
PERF-05	Memoisation	useMemo for filtered/sorted country lists; React.memo on CountryCard to prevent unnecessary re-renders.
PERF-06	Production Build	Vite tree-shakes unused code; assets are minified and compressed (Gzip/Brotli) on Vercel/Netlify.
PERF-07	Pagination	CountryGrid renders 20 cards per page; remaining loaded on scroll/button click.

12. Deployment Design

12.1 Build Pipeline

Stage	Tool	Action
1	**Developer Machine**	Code written in VS Code; linted with ESLint; formatted with Prettier.
2	**Git**	Feature branches merged to main via pull request.
3	**GitHub Actions (CI)**	Automated: npm install, npm run lint, npm test, npm run build.
4	**Vite Build**	Produces dist/ folder with minified HTML, CSS, JS bundles.
5	**Vercel / Netlify (CD)**	Auto-deploys on push to main; serves static files from CDN edge nodes.

12.2 Directory Structure

```
global-explorer/
  ├── public/              # Static assets (favicon, robots.txt)
  ├── src/
  |    ├── components/      # Reusable UI components
  |    |    ├── Navbar/
  |    |    ├── CountryCard/
  |    |    ├── SearchBar/
  |    |    ├── RegionFilter/
  |    |    ├── ThemeToggle/
  |    |    └── ErrorBoundary/
  |    ├── pages/           # Route-level page components
  |    |    ├── HomePage.jsx
  |    |    ├── DetailPage.jsx
  |    |    ├── ComparePage.jsx
  |    |    └── FavoritesPage.jsx
  |    ├── hooks/           # Custom React hooks
  |    |    ├── useCountries.js
  |    |    ├── useCountry.js
  |    |    ├── useFavorites.js
  |    |    └── useDebounce.js
  |    ├── services/        # API and storage helpers
  |    |    ├── apiService.js
  |    |    └── storageService.js
  |    ├── context/         # React Context providers
  |    |    └── ThemeContext.jsx
  |    ├── styles/          # Global CSS / theme variables
  |    ├── App.jsx          # Root component
  |    └── main.jsx         # ReactDOM entry point
  ├── .eslintrc.json
  ├── vite.config.js
  ├── package.json
  └── README.md
```

13. Testing Design

Test Type	Tool	Scope	Pass Criteria
Unit Tests	Jest + React Testing Library	Custom hooks (useCountries, useFavorites), storageService functions.	All functions return expected output; edge cases covered.
Component Tests	React Testing Library	CountryCard, SearchBar, Navbar — render and interaction tests.	Components render correctly; user events produce correct state changes.
Integration Tests	Jest / MSW (Mock Service Worker)	API service with mocked REST responses; data flow from fetch to render.	Mocked API data displays correctly in UI components.
End-to-End Tests	Cypress	Search flow, detail page navigation, compare flow, favorites persistence.	All user journeys complete without errors.
Accessibility Tests	axe-core (automated)	All pages checked for WCAG 2.1 AA violations.	Zero high-severity accessibility violations.
Performance Tests	Lighthouse (CI)	Performance, Accessibility, Best Practices, SEO scores.	Performance ≥ 90; Accessibility ≥ 90.

14. Design Traceability Matrix

Each SRS requirement is mapped to its corresponding design element in this document:

SRS Req.	Requirement Summary	Design Element	Section
FR-01	Country search with autocomplete	SearchBar (MOD-09), useDebounce hook	§4.2, §7.1, §8.2
FR-02	Display all countries as cards	CountryGrid + CountryCard (MOD-04/05)	§3.2, §4.1
FR-03	Region filter dropdown	RegionFilter (MOD-10), useCountries hook	§4.1, §8.2
FR-04	Country detail page	DetailPage (MOD-06), apiService.getCountryByCode	§4.1, §5.1, §7.1
FR-05	Side-by-side comparison	ComparePage + CompareTable (MOD-07)	§4.1, §6.2.3
FR-06	Favorites with persistence	useFavorites hook, storageService (MOD-13)	§5.2, §7.2, §8.2
FR-07	Dark/light theme toggle	ThemeContext (MOD-15), ThemeToggle (MOD-11)	§6.3, §8.1
FR-08	Graceful error handling	ErrorBoundary, apiService error states	§9.1, §9.2
NFR-01	Load time < 3 seconds	Lazy loading, single /all fetch, pagination	§11
NFR-02	WCAG 2.1 AA accessibility	Design tokens (contrast), touch targets 44px	§6.1, §13
NFR-03	Responsive 320–2560 px	Mobile-first CSS, CSS Grid breakpoints	§6.1
NFR-04	HTTPS only	apiService enforces HTTPS base URL	§10

15. Conclusion

This Software System Design document presents a complete, coherent, and implementable design for Global Explorer. The layered component architecture cleanly separates concerns between presentation, logic, API communication, and persistence. The design is intentionally lightweight — leveraging React's built-in primitives (hooks, context, lazy loading) rather than heavyweight external dependencies — making it well-suited for a single-developer academic project within a 12-week timeline.

Key design strengths include:

- Clear module boundaries with well-defined interfaces simplify implementation and testing.
- Custom hooks encapsulate all async and storage logic, keeping components declarative.
- The traceability matrix guarantees full coverage of SRS requirements.
- Security, performance, and accessibility are treated as first-class design concerns, not afterthoughts.

Upon following this design, the resulting application will be a polished, responsive, and deployable web product that demonstrates competence in modern frontend engineering and fulfils the academic objectives of the Final Year Project.

— Alina Farooq | S25BINFT7M04067